BÁO CÁO THỰC HÀNH LAB 01

Cấu hình Kernel & Khởi động Hệ thống Linux Nhúng (ARMv7)

Mục 1: Thông tin sinh viên & Môi trường thực hiện

Họ và tên:	Vũ Tùng Minh
Mã số sinh viên (MSSV):	SE201043
Lớp chuyên ngành:	AS2105 (Môn: LOS201 - Embedded Linux Systems)
Thời gian thực hiện:	12/09/2026 - 19/09/2026
Môi trường thực nghiệm:	CachyOS Linux x86_64, QEMU v11.1.1, Ubuntu 22.04 LTS (Podman Container)

Mục 2: Boot Sequence Diagram

Chuỗi khởi động của Linux trên kiến trúc ARMv7 (Versatile Express Cortex-A9) từ lúc cấp nguồn tới khi mở shell tương tác:

[1. Power ON / Reset]
↓ Cấp nguồn phần cứng, CPU thực thi lệnh tại Reset Vector
[2. ROM Bootloader]
↓ Khởi tạo xung nhịp cơ bản, SRAM, nạp U-Boot vào DRAM
[3. U-Boot (BL2)]
↓ Khởi tạo DRAM, UART0, nạp zImage, DTB, initramfs vào RAM; gọi 'bootz'
[4. Linux Kernel (zImage)]
↓ Giải nén kernel, chạy start_kernel(), nạp driver, bung initramfs vào rootfs
[5. Userspace Init (/sbin/init)]
↓ Đọc /etc/inittab, chạy /etc/init.d/rcS mount /proc, /sys, /tmp
[6. BusyBox Shell (/bin/sh)]

Bảng chi tiết các giai đoạn khởi động:

Bước	Thành phần	Công việc chính	Dữ liệu bàn giao tiếp theo
1	Phần cứng SoC / Power	PMIC cấp nguồn ổn định, bộ tạo dao động cấp clock, nhả tín hiệu reset, CPU nhảy về địa chỉ Reset Vector.	Chuyển quyền điều khiển cho Boot ROM.
2	ROM Bootloader	Khởi tạo clock tối thiểu, SRAM nội bộ, đọc cấu hình chân boot và nạp U-Boot vào DRAM.	Nhảy tới địa chỉ nạp U-Boot trong RAM (0x60000000).
3	U-Boot 2022.04	Khởi tạo DRAM, cổng nối tiếp UART0, nạp zImage, DTB và initramfs vào các địa chỉ RAM tương ứng. Đọc cấu hình bootargs.	Ghi tham số bootargs vào DTB, đặt con trỏ DTB vào thanh ghi r2, gọi lệnh bootz.

4	Linux Kernel 5.15	Tự giải nén vmlinux, start_kernel() khởi tạo MMU, ngắt GIC, timer, driver UART PL011. Bung initramfs vào rootfs tmpfs.	Chuyển sang userspace, gọi kernel_execve('/sbin/init') với PID 1.
5	BusyBox Init	Đọc file /etc/inittab, thực thi kịch bản /etc/init.d/rcS để mount các filesystem ảo (/proc, /sys, /tmp).	Mở terminal điều khiển /dev/console và spawn tiến trình /bin/sh.
6	BusyBox Shell	Khởi tạo shell tương tác, in dòng nhắc '/ # ' ra terminal.	Sẵn sàng nhận và thực thi lệnh của người dùng.

Mục 3: Log thực hành và phân tích ảnh chụp

Chi tiết các bước thực hiện trên môi trường máy chủ CachyOS (container Ubuntu 22.04) và giả lập QEMU ARM:

Phần A: Cấu hình và biên dịch Linux Kernel

```
root@0dab86407b4c:~/embedded_lab1# arm-linux-gnueabihf-gcc --version
arm-linux-gnueabihf-gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

root@0dab86407b4c:~/embedded_lab1#
```

Hình 1: Kiểm tra phiên bản cross-compiler arm-linux-gnueabihf-gcc (Ubuntu 11.4.0)

Compiler arm-linux-gnueabihf-gcc 11.4.0 hỗ trợ kiến trúc ARMv7 với hard-float ABI, sử dụng phần cứng FPU của chip Cortex-A9 để tính toán dấu phẩy động.

```
root@0dab86407b4c:~/embedded_lab1/kernel/linux-5.15# make vexpress_defconfig
  HOSTCC  scripts/basic/fixdep
  HOSTCC  scripts/kconfig/conf.o
  HOSTCC  scripts/kconfig/confdata.o
  HOSTCC  scripts/kconfig/expr.o
  LEX     scripts/kconfig/lexer.lex.c
  YACC    scripts/kconfig/parser.tab.[ch]
  HOSTCC  scripts/kconfig/lexer.lex.o
  HOSTCC  scripts/kconfig/menu.o
  HOSTCC  scripts/kconfig/parser.tab.o
  HOSTCC  scripts/kconfig/preprocess.o
  HOSTCC  scripts/kconfig/symbol.o
  HOSTCC  scripts/kconfig/util.o
  HOSTLD  scripts/kconfig/conf
#
# configuration written to .config
#
root@0dab86407b4c:~/embedded_lab1/kernel/linux-5.15#
```

Hình 2: Nạp cấu hình mặc định vexpress_defconfig cho bo mạch Versatile Express

Lệnh make vexpress_defconfig tạo file .config chứa cấu hình phần cứng tiêu chuẩn cho dòng bo mạch Versatile Express Cortex-A9, bao gồm CONFIG_ARCH_VEXPRESS=y, CONFIG_CPU_V7=y và driver UART PL011.

```
  SHIPPED arch/arm/boot/compressed/ashldi3.S
  SHIPPED arch/arm/boot/compressed/bswapsdi2.S
  AS      arch/arm/boot/compressed/hyp-stub.o
  AS      arch/arm/boot/compressed/lib1funcs.o
  AS      arch/arm/boot/compressed/ashldi3.o
  AS      arch/arm/boot/compressed/bswapsdi2.o
  AS      arch/arm/boot/compressed/piggy.o
  LD      arch/arm/boot/compressed/vmlinux
  OBJCOPY arch/arm/boot/zImage
  Kernel: arch/arm/boot/zImage is ready
root@0dab86407b4c:~/embedded_lab1/kernel/linux-5.15#
```

Hình 3: Quá trình biên dịch kernel hoàn tất thành công

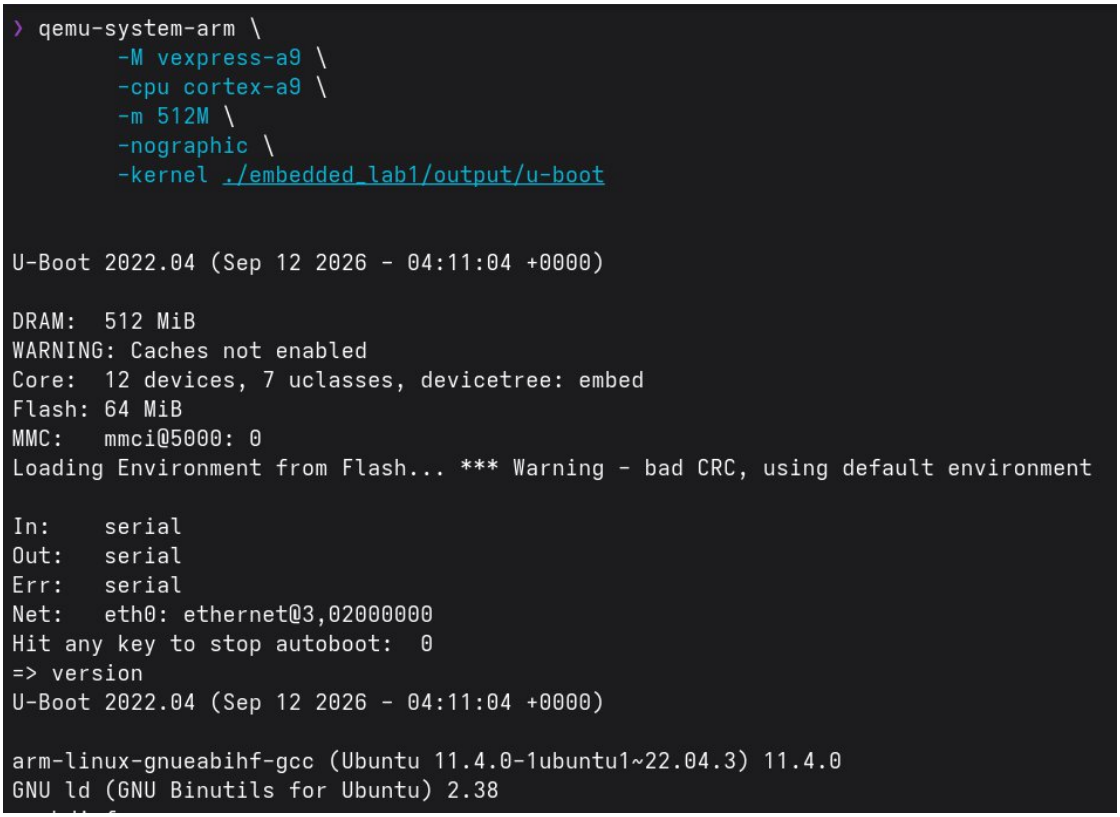
Sau khi bật hỗ trợ initramfs, serial driver và SMP trong menuconfig, lệnh make -j$(nproc) zImage dtbs modules hoàn tất và tạo file ảnh kernel nén tại arch/arm/boot/zImage.

```
ls -lh arch/arm/boot/zImage arch/arm/boot/dts/vexpress-v2p-ca9.dtb
arm-linux-gnueabihf-readelf -A arch/arm/boot/compressed/vmlinux | grep -i 'arch\|cpu'
-rw-r--r-- 1 root root  14K Sep 12 04:06 arch/arm/boot/dts/vexpress-v2p-ca9.dtb
-rwxr-xr-x 1 root root 4.8M Sep 12 04:08 arch/arm/boot/zImage
  Tag_CPU_name: "7-A"
  Tag_CPU_arch: v7
  Tag_CPU_arch_profile: Application
  Tag_FP_arch: VFPv2
  Tag_CPU_unaligned_access: v6
root@0dab86407b4c:~/embedded_lab1/kernel/linux-5.15#
```

Hình 4: Kiểm tra định dạng nhị phân và kích thước file zImage

File zImage có dung lượng 4.8 MB. Lệnh file và readelf -A xác nhận file mang định dạng ARM 32-bit EABI5, Tag_CPU_arch: v7, tương thích với kiến trúc Cortex-A9.

Phần B: Khởi động và cấu hình U-Boot trên QEMU



Hình 5: Khởi động U-Boot trên QEMU và kiểm tra phiên bản bằng lệnh version

QEMU vexpress-a9 nạp firmware U-Boot 2022.04 thành công. Lệnh version xác nhận U-Boot chạy trên CPU Cortex-A9 r3p0 và bo mạch V2P-CA9.

```
=> bdinfo
boot_params = 0x60002000
DRAM bank   = 0x00000000
-> start    = 0x60000000
-> size     = 0x20000000
DRAM bank   = 0x00000001
-> start    = 0x80000000
-> size     = 0x00000004
flashstart  = 0x40000000
flashsize   = 0x04000000
flashoffset = 0x00000000
baudrate    = 38400 bps
relocaddr   = 0x7ff6b000
reloc off   = 0x1f76b000
Build       = 32-bit
current eth = ethernet@3,02000000
ethaddr     = 52:54:00:12:34:56
IP addr     = <NULL>
fdt_blob    = 0x7ffe0120
new_fdt     = 0x00000000
fdt_size    = 0x00000000
lmb_dump_all:
 memory.cnt  = 0x1
 memory[0]      [0x60000000-0x80000003], 0x20000004 bytes flags: 0
 reserved.cnt  = 0x2
 reserved[0]    [0x4c000000-0x4c7fffff], 0x00800000 bytes flags: 4
 reserved[1]    [0x7fb26c88-0x7fffffff], 0x004d9378 bytes flags: 0
devicetree  = embed
arch_number = 0x000008e0
TLB addr    = 0x7fff0000
irq_sp      = 0x7fb2aec0
sp start    = 0x7fb2aeb0
Early malloc usage: 370 / 400
```

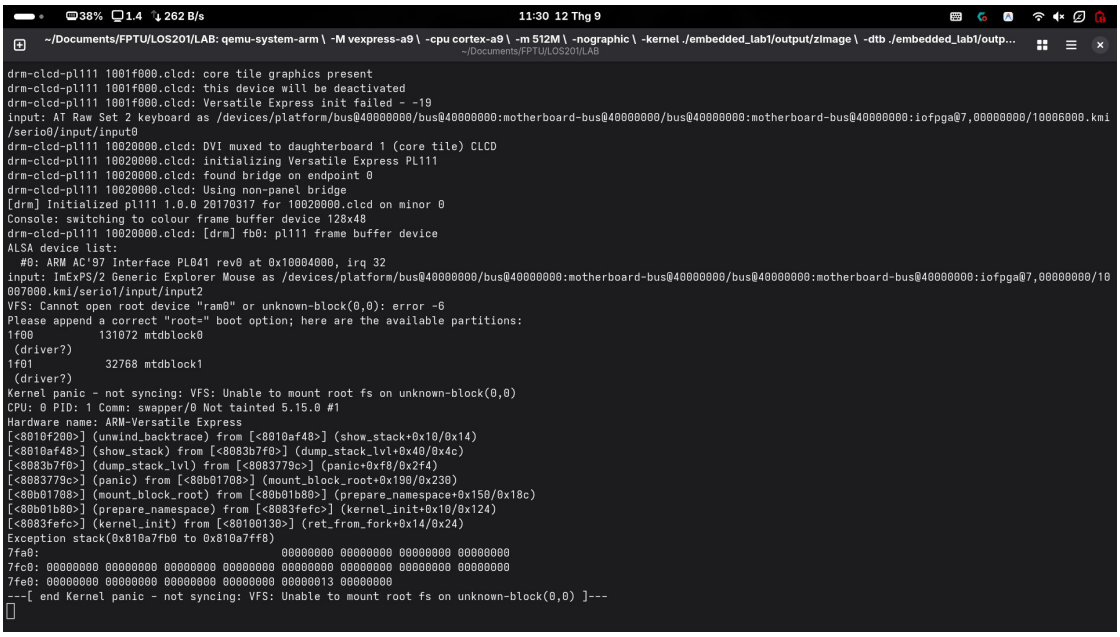
Hình 6: Thông tin cấu trúc bộ nhớ vật lý qua lệnh bdinfo

Lệnh bdinfo hiển thị bản đồ bộ nhớ của bo mạch: DRAM bắt đầu từ địa chỉ 0x60000000 với dung lượng 512 MiB. NOR Flash nằm tại 0x40000000. Đây là căn cứ để đặt địa chỉ nạp cho kernel và initramfs.

```
=> setenv bootargs 'console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M'
=> setenv bootcmd 'bootz 0x60100000 0x64000000 0x60000000'
=>  printenv bootcmd
bootcmd=bootz 0x60100000 0x64000000 0x60000000
=>
```

Hình 7: Thiết lập biến môi trường bootargs và bootcmd

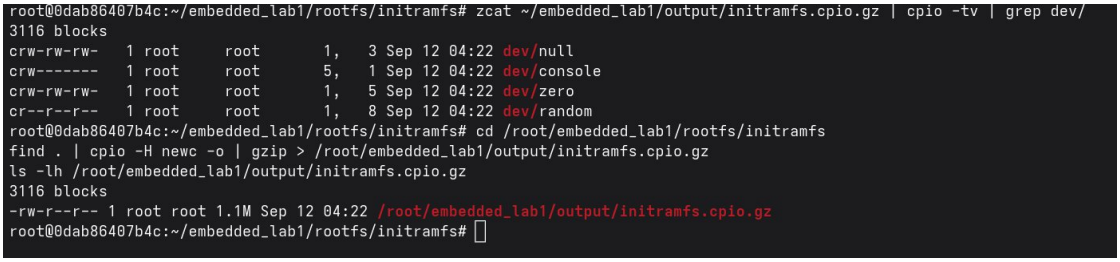
Biến bootargs được cấu hình để xuất console ra cổng ttyAMA0 tốc độ 115200, chọn rootfs trên RAM (/dev/ram0) và đặt giới hạn RAM 512M. Lệnh bootcmd nạp zImage tại 0x60100000, initramfs tại 0x64000000 và DTB tại 0x60000000.



Hình 8: Thử nghiệm lỗi Kernel Panic khi khởi động thiếu Root Filesystem

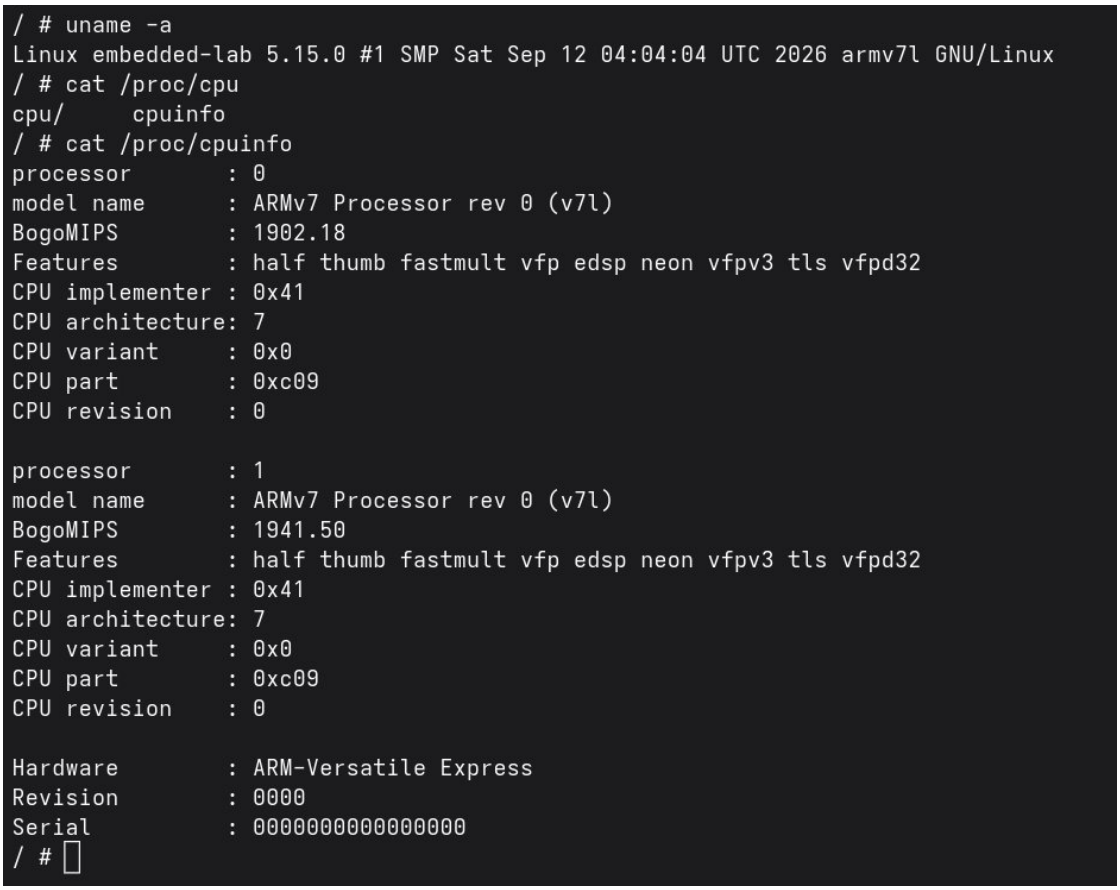
Khi nạp kernel và DTB nhưng không truyền initramfs, kernel khởi tạo xong phần cứng nhưng dừng lại ở bước mount rootfs cuối cùng, báo lỗi 'Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(1,0)'. Thử nghiệm này xác nhận hệ thống bắt buộc phải có root filesystem để chuyển sang userspace.

Phần C: Đóng gói initramfs và khởi động hệ thống hoàn chỉnh



Hình 9: Kiểm tra cấu trúc archive initramfs.cpio.gz và các character device nodes

File initramfs.cpio.gz (1.0 MB) được kiểm tra bằng lệnh zcat ... | cpio -tv. Các file thiết bị dev/console (major 5, minor 1), dev/null (major 1, minor 3), dev/zero và dev/random đều mang thuộc tính character device ('crw-...'), được đóng gói chuẩn xác thông qua fakeroot.



Hình 10: Khởi động hệ thống hoàn chỉnh lên BusyBox Shell và xác minh

Hệ thống khởi động qua QEMU vexpress-a9, chạy kịch bản /etc/init.d/rcS và hiển thị dấu nhắc lệnh BusyBox ('/ # '). Các lệnh xác minh hệ thống:
• uname -a: Linux embedded-lab 5.15.0 ... armv7l GNU/Linux.
• cat /proc/cpuinfo: Bo mạch ARM-Versatile Express, vi xử lý Cortex-A9 (part 0xc09, implementer 0x41), nhận diện đủ 2 CPU cores nhờ bật SMP.
• cat /proc/meminfo: Tổng bộ nhớ hệ thống quản lý đạt khoảng 489 MB trên tổng 512 MB cấp phát.
• mount: Các filesystem ảo /proc, /sys và /tmp đã được mount đúng kịch bản.
• ps: Tiến trình /sbin/init (PID 1) và /bin/sh (PID 36) đang chạy bình thường.

Mục 4: Trả lời câu hỏi phân tích (Mục VI)

Câu 1. So sánh toolchain arm-linux-gnueabi và arm-linux-gnueabihf. Tại sao Cortex-A9 phải dùng gnueabihf? Điều gì xảy ra nếu build kernel bằng gnueabi rồi chạy trên Cortex-A9?

Sự khác biệt nằm ở cách thức ABI xử lý các phép toán số thực (floating-point):

• arm-linux-gnueabi (Soft-Float ABI): Đối số số thực khi gọi hàm được truyền qua các thanh ghi số nguyên đa năng (r0-r3) hoặc đưa lên stack. Nếu chip không có phần cứng FPU, compiler sẽ chèn các hàm thư viện phần mềm (__aeabi_fadd, __aeabi_dmul) để tính toán mô phỏng.

• arm-linux-gnueabihf (Hard-Float ABI): Ký tự 'hf' chỉ định phần cứng FPU. Compiler sinh mã máy sử dụng trực tiếp các thanh ghi số thực phần cứng (s0-s31, d0-d15 của VFP/NEON) để truyền tham số và tính toán trực tiếp.

Lý do dùng gnueabihf cho Cortex-A9: Cortex-A9 tích hợp sẵn bộ đồng xử lý FPU VFPv3 và NEON. Dùng gnueabihf giúp tận dụng trực tiếp tập lệnh phần cứng của chip, tăng tốc độ xử lý các phép tính số thực so với việc dùng CPU mô phỏng bằng phần mềm.

Khi build kernel bằng gnueabi và chạy trên Cortex-A9: Kernel Linux vẫn khởi động và hoạt động bình thường. Trong không gian nhân (kernel-space), Linux không sử dụng các phép tính số thực để tránh chi phí lưu và phục hồi 32 thanh ghi FPU trong mỗi lần chuyển ngữ cảnh (context switch) hoặc xử lý ngắt. File arch/arm/Makefile trong kernel luôn tự đặt cờ -mfloat-abi=soft. Tuy nhiên, ở tầng userspace, ứng dụng biên dịch dạng hard-float sẽ lỗi nếu liên kết với thư viện glibc soft-float do lệch chuẩn ABI.

Câu 2. Giải thích tại sao memory map của vexpress-a9 bắt đầu từ 0x60000000 thay vì 0x00000000 như Versatile PB. Ảnh hưởng tới bootcmd và hậu quả nếu đặt sai địa chỉ?

Nguyên nhân: Bo mạch Versatile PB đời cũ đặt bộ điều khiển SDRAM ngay từ địa chỉ 0x00000000. Ngược lại, Versatile Express (vexpress-a9) sử dụng kiến trúc CoreTile hiện đại hơn: toàn bộ dải địa chỉ từ 0x00000000 đến 0x5FFFFFFF (1.5 GB đầu tiên) được dành cho bus ngoại vi AMBA, các thanh ghi điều khiển thiết bị, SRAM và hai chip NOR Flash (ánh xạ từ 0x40000000). Bộ nhớ RAM vật lý trên Daughterboard bắt đầu được ánh xạ từ địa chỉ 0x60000000 trở lên.

Ảnh hưởng đến bootcmd: U-Boot bắt buộc phải nạp kernel, DTB và initramfs vào vùng RAM từ 0x60000000 trở lên. Cụ thể:
• DTB: nạp tại 0x60000000 (đầu vùng RAM, dung lượng nhỏ ~14 KB).
• zImage: nạp tại 0x60100000 (offset 1 MB tính từ đầu RAM để tránh ghi đè lên DTB).
• initramfs: nạp tại 0x64000000 (offset 64 MB, tạo khoảng trống an toàn cho kernel tự giải nén).

Hậu quả nếu đặt sai địa chỉ:
• Nếu đặt địa chỉ dưới 0x60000000 (như 0x00000000 của Versatile PB): CPU truy xuất vào vùng không có RAM hoặc vùng Flash chỉ đọc, gây lỗi ngoại lệ Data Abort và treo hệ thống ngay khi boot.
• Nếu các địa chỉ nạp đặt quá sát nhau (ví dụ nạp initramfs tại 0x60200000): Quá trình giải nén zImage sẽ bung mã máy vmlinux (~15 MB) đè lên vùng nhớ của DTB hoặc initramfs, dẫn đến lỗi hỏng dữ liệu và phát sinh Kernel Panic.

Câu 3. Giải thích ý nghĩa của MIDR 0x410fc090 trong boot log. Part number 0xc09 tương ứng với CPU nào?

Thanh ghi MIDR (Main ID Register) 32-bit gồm các trường dữ liệu theo tài liệu ARM Architecture Reference Manual:

Trường dữ liệu	Bit Fields	Giá trị	Ý nghĩa phân tích
Implementer	[31:24]	0x41 (ASCII 'A')	Hãng thiết kế ARM Limited.
Variant	[23:20]	0x0	Phiên bản thiết kế chính: r0.
Architecture	[19:16]	0xF	Kiến trúc CPUID (ARMv7 trở lên).

Part Number	[15:4]	0xC09	Mã định danh lõi: ARM Cortex-A9.
Revision	[3:0]	0x0	Phiên bản vá lỗi phụ: p0 (Tổng hợp: r0p0).

Part number 0xC09 tương ứng với CPU ARM Cortex-A9 (dòng lõi 32-bit hiệu năng cao thuộc kiến trúc ARMv7-A). Để so sánh: 0xC07 = Cortex-A7, 0xC08 = Cortex-A8, 0xC0F = Cortex-A15, 0xD03 = Cortex-A53.

Câu 4. Tại sao cần build BusyBox dạng static linking trong initramfs? Nếu build dynamic linking, cần copy ít nhất 3 file .so nào và chúng nằm ở đâu trong toolchain?

Lý do dùng static linking trong initramfs tối giản:

• Tính độc lập: Mã thực thi của thư viện C (libc) được nhúng trực tiếp vào file nhị phân /bin/busybox. Kernel chỉ cần nạp duy nhất file này vào RAM là có thể chạy ngay, không cần thêm dynamic linker hay bất kỳ file shared library (.so) nào.

• Tránh lỗi khởi động: Nếu thiếu file .so hoặc đặt sai đường dẫn linker (/lib/ld-linux-armhf.so.3), kernel sẽ báo lỗi kernel_execve: -2 (No such file or directory) và Kernel Panic ngay lập tức.

• Giảm dung lượng: Giúp initramfs gọn nhẹ (~1 MB), tiết kiệm RAM và không cần quản lý dependencies phức tạp.

Danh sách 3 file .so bắt buộc nếu dùng dynamic linking:

Tên file .so	Vai trò kỹ thuật	Đường dẫn trong Toolchain (Sysroot)
ld-linux-armhf.so.3	Dynamic Linker / Program Loader. Thành phần nạp các file .so khác vào RAM khi ứng dụng khởi động.	/usr/arm-linux-gnueabihf/lib/ld-linux-armhf.so.3
libc.so.6 (glibc)	Thư viện C chuẩn, cung cấp các hàm hệ thống cơ bản như open, read, write, fork, exec.	/usr/arm-linux-gnueabihf/lib/libc.so.6
libm.so.6	Thư viện toán học, cung cấp các hàm tính toán số thực cho các lệnh trong BusyBox.	/usr/arm-linux-gnueabihf/lib/libm.so.6

Câu 5. Khi bật tùy chọn SMP (-smp 2), Kernel cần khởi tạo thêm những gì so với uniprocessor? Quan sát và phân tích ít nhất 2 dòng boot log thay đổi.

Các thành phần khởi tạo thêm khi bật SMP (Symmetric Multi-Processing):

Trong hệ thống đa lõi, CPU 0 (Boot Strap Processor - BSP) khởi động trước, sau đó đánh thức CPU 1 (Secondary Core). Kernel cần thiết lập thêm:
• Per-CPU Memory: Cấp phát vùng nhớ riêng cho từng CPU để lưu trạng thái và hàng đợi tác vụ (runqueue).
• Inter-Processor Interrupts (IPI): Cơ chế ngắt liên CPU để đồng bộ hóa bộ nhớ cache và TLB flush giữa hai lõi.
• Spinlocks: Kích hoạt các cơ chế khóa đồng bộ hạt nhân để tránh xung đột tài nguyên dùng chung.
• Load Balancer: Bộ lập lịch cân bằng tải tiến trình giữa hai lõi CPU.

• Đánh thức lõi thứ hai: CPU 0 ghi địa chỉ nhảy thực thi vào thanh ghi phần cứng (SYS_FLAGS) rồi phát lệnh SEV (Send Event) để kích hoạt CPU 1.

So sánh sự khác biệt trong log khởi động giữa -smp 1 và -smp 2:

Log khi dùng -smp 1 (Đơn nhân)	Log khi dùng -smp 2 (Đa nhân)	Ý nghĩa kỹ thuật
Brought up 1 CPUs	smp: Bringing up secondary CPUs ... CPU1: Booted secondary processor	Xác nhận CPU 0 phát ngắt IPI/SEV đánh thức thành công CPU 1 vào hệ thống.
SMP: Total of 1 processors activated	smp: Brought up 1 node, 2 CPUs SMP: Total of 2 processors activated	Kernel xác nhận cả 2 lõi CPU đều hoạt động ở chế độ SVC mode với tổng BogoMIPS tăng gấp đôi (~1825 so với ~910).
/sys/.../possible: 0	/sys/.../possible: 0-1 cat /proc/cpuinfo hiện core 0 & 1	Kernel công bố ra userspace quản lý đồng thời hai CPU logic hoạt động song song.

Mục 5: Nhận xét & Khó khăn thực tế gặp phải

Quá trình triển khai bài lab trên CachyOS Linux (sử dụng container Podman Ubuntu 22.04) phát sinh 3 sự cố kỹ thuật cụ thể:

1. Xung đột ABI cờ biên dịch dẫn đến lỗi Assembler ARMv5T

Hiện tượng: Khi thiết lập export KCFLAGS="-march=armv7-a -mfpu=vfpv3 -mfloat-abi=hard" theo tài liệu hướng dẫn, lệnh make kernel báo lỗi xung đột cờ: '-mfloat-abi=soft and -mfloat-abi=hard may not be used together'. Sau khi unset biến KCFLAGS, quá trình dịch code inline assembly lại tiếp tục báo lỗi: 'selected processor does not support dsb st in ARM mode'.

Nguyên nhân: Kernel Linux trong arch/arm/Makefile tự ép cờ -mfloat-abi=soft do không dùng FPU trong nhân. Compiler arm-linux-gnueabihf-gcc mặc định lại là hard-float. Khi Kbuild gọi macro cc-option kiểm tra cờ -march=armv7-a, lệnh kiểm tra bị lỗi vì thiếu FPU tương thích. Kbuild âm thầm chuyển sang phương án dự phòng là kiến trúc ARMv5T (-march=armv5t). Khi biên dịch các lệnh rào cản bộ nhớ của ARMv7 (dsb, dmb, isb), assembler từ chối vì kiến trúc ARMv5T không có các lệnh này.

Cách xử lý: Thiết lập tường minh biến CC cho target compiler: export CC="arm-linux-gnueabihf-gcc -mfloat-abi=soft". Khi đó cc-option kiểm tra thành công, Kbuild gán đúng kiến trúc ARMv7-A và kernel được biên dịch bình thường.

2. Lỗi quyền tạo file thiết bị trong rootless container làm treo boot

Hiện tượng: Khi chạy lệnh mknod -m 600 dev/console c 5 1 trong container, hệ thống báo lỗi 'Operation not permitted'. Nếu dùng lệnh touch tạo file rỗng thay thế, QEMU boot đến dòng 'random: fast init done' rồi dừng lại, không hiển thị thông báo khởi động và không vào được shell.

Nguyên nhân: Chế độ rootless podman chặn quyền CAP_MKNOD vì lý do bảo mật của máy host. Việc thay thế bằng file thông thường khiến kernel gán stdin/stdout/stderr (fd 0, 1, 2) vào file rỗng trong RAM thay vì cổng nối tiếp ttyAMA0. Toàn bộ thông tin xuất ra màn hình và tương tác shell bị đẩy vào file rỗng đó, làm tiến trình init không nhận được input và dừng lại.

Cách xử lý: Xóa sạch các file rỗng cũ trong thư mục dev/ và dùng công cụ fakeroot để đóng gói: fakeroot bash -c 'mknod ... && find . | cpio -H newc -o | gzip > initramfs.cpio.gz'. Fakeroot lưu thông tin major/minor trong bộ nhớ và ghi nhận đúng định dạng character device vào file archive.

3. Phím tắt thoát QEMU bị terminal của IDE chiếm dụng

Hiện tượng: Khi chạy QEMU ở chế độ -nographic, tổ hợp phím thoát Ctrl+A rồi X không có tác dụng, terminal tự động bôi đen toàn bộ văn bản (Select All).

Nguyên nhân: Terminal của trình soạn thảo IDE bắt chặn phím Ctrl+A để thực hiện chức năng Select All, khiến mã byte 0x01 không được gửi đến tiến trình QEMU.

Cách xử lý: Thêm tùy chọn -echr 2 vào lệnh khởi động QEMU để đổi phím escape mặc định từ Ctrl+A sang Ctrl+B. Khi cần thoát chỉ cần bấm Ctrl+B rồi X.